CS326 – Systems Security

Lecture 11

Control-flow Attacks

Elias Athanasopoulos
athanasopoulos.elias@ucy.ac.cy

Functions

- Software is composed by several functions
  - `main()`, `printf()`, `malloc()`, `create_user()`, etc.
- Functions allow code re-use
  - Whenever you want to display a message you simply call `printf`
- In a program a function can call a function, and then another function
  - All this function chaining is called **control flow**

The life of a function

- Whenever a function is called, the control flow of the program is changed
  - We need to do this transparently
  - Once the function is finished the control flow should be resumed
- Functions may take arguments
- Functions may return data
- Functions may create local data

Vocabulary

- When a function `foo` is called
  - `foo` is the callee
  - The address that called the function is called **call site** (or caller)

The stack

- Functions need memory for their work
  - This is the stack
- This memory is for short lived data
  - Once the function is finished we can get rid of the data involved
- Architecture dependent
  - The main idea does not change
- The stack may hold several things
  - Function arguments, the return address, the old frame pointer, local arguments

Stack of Intel (32-bit)

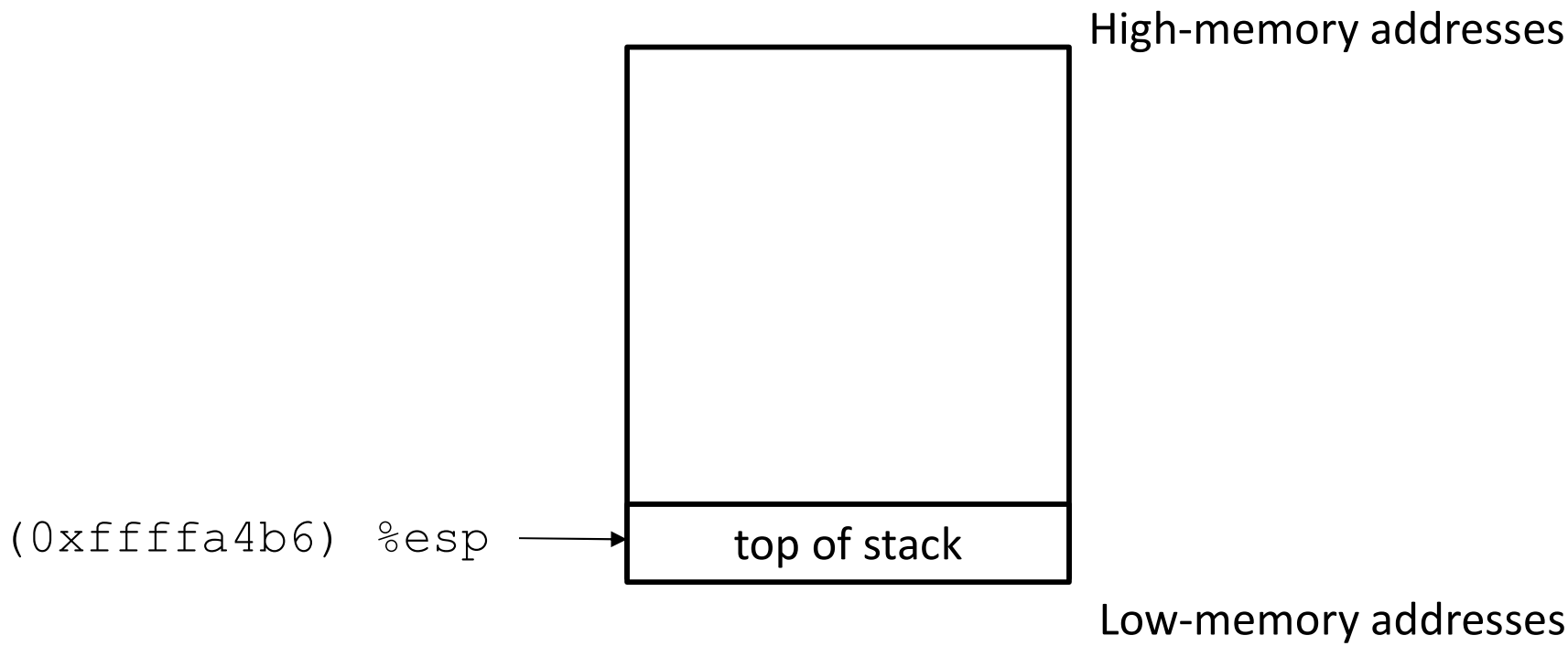
- The stack grows from higher-memory addresses to lower-memory addresses
  - It is like the stack is flipped upside down
- The top of the stack is always kept in a hardware register (`%esp`)
- Each function creates a new **stack frame** upon executing
  - A virtual portion inside the stack
  - The stack frame is destroyed once the functions is finished
- The top of the stack frame is kept in a hardware register (`%ebp`)

Stack insertion

- `push %eax`

  `sub 0x4, %esp`

  `mov %eax, (%esp)`

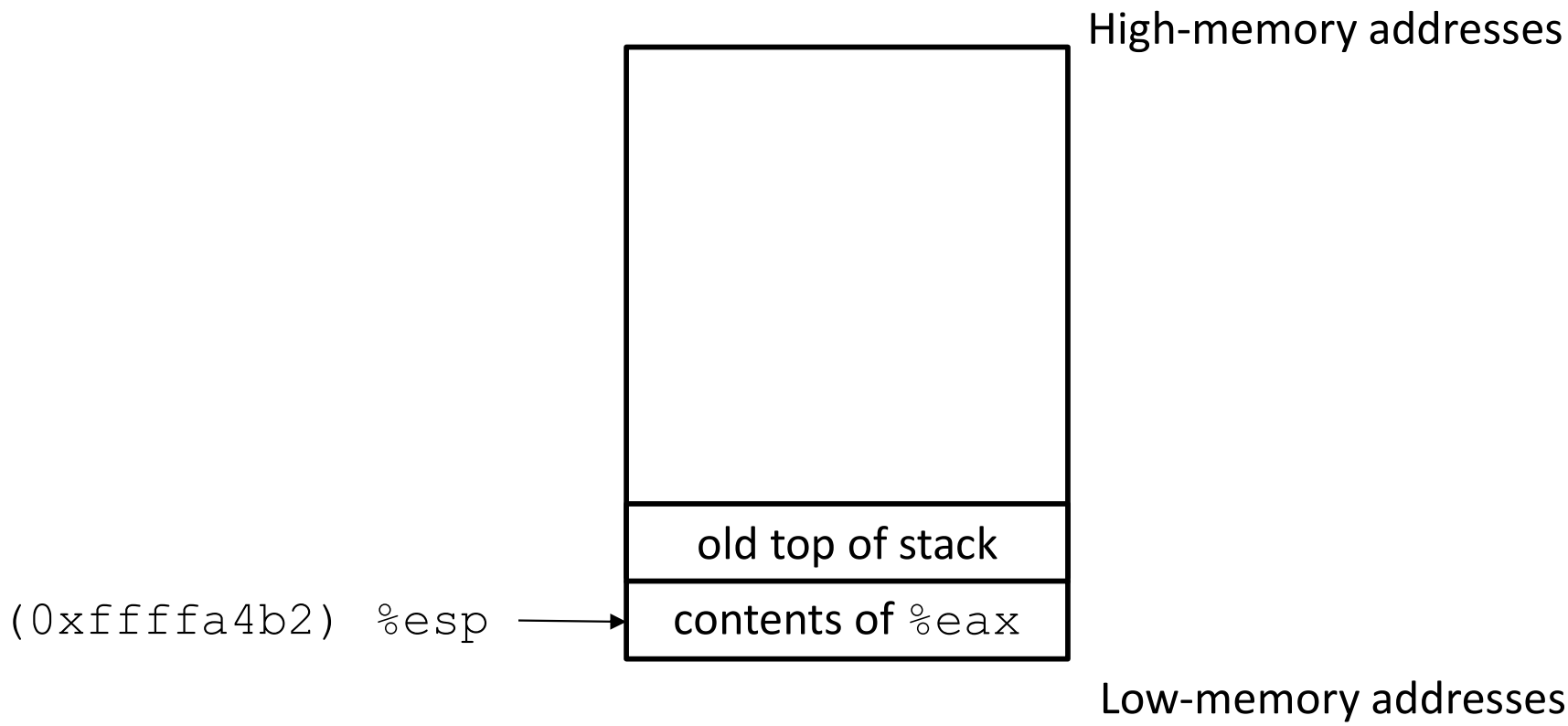


Stack insertion

- `push %eax`

  `sub 0x4, %esp`

  `mov %eax, (%esp)`

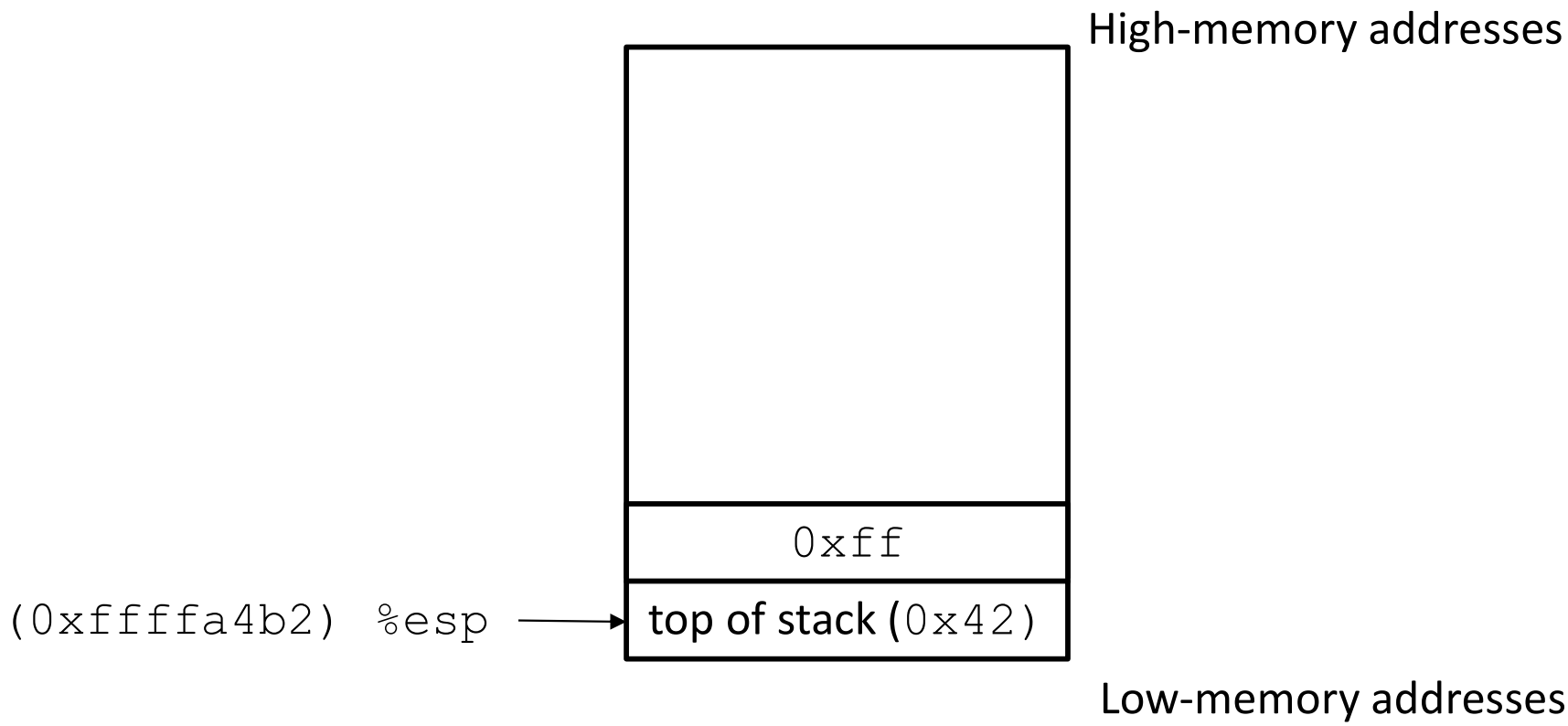


Stack deletion

- `pop %eax`

  `mov %(esp), %eax`

  `add 0x4, %esp`

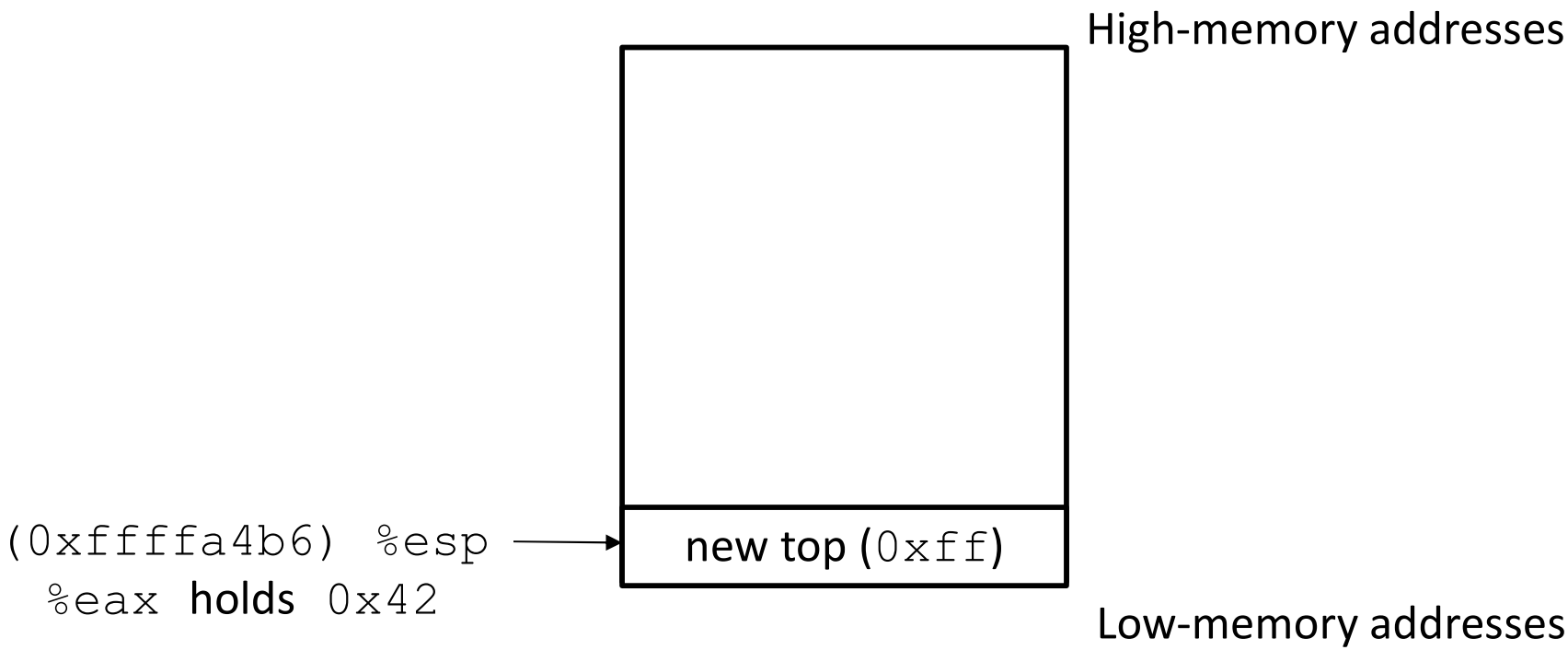


Stack deletion

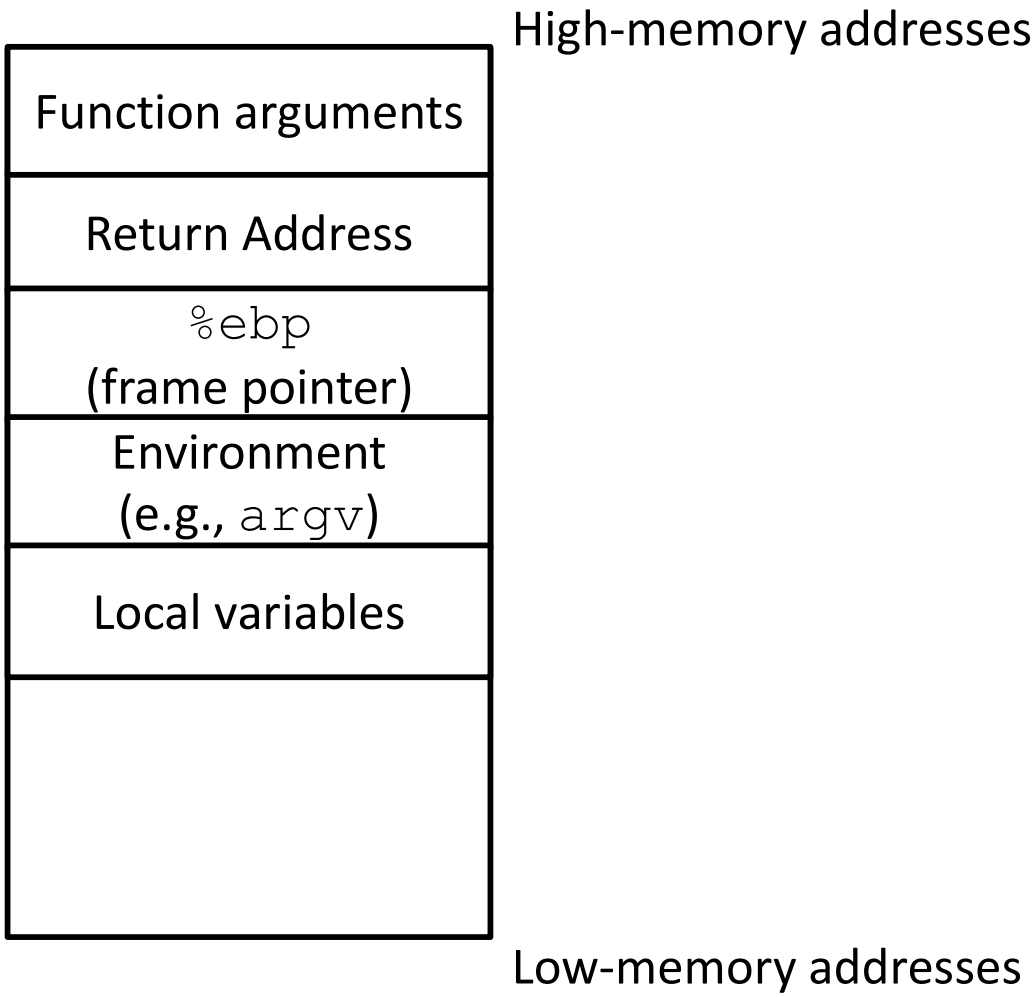
- `pop %eax`

  `mov %(esp), %eax`

  `add 0x4, %esp`



Stack frame in Intel 32-bit



Endianness

- Assume the 32-bit word: `0x0A0B0C0D`
- Two possible ways to store it in memory

Little Endian (Intel)

```
0x0D 0x0C 0x0B 0x0A

Low Mem          High Mem
```

Big Endian (Motorola)

```
0x0A 0x0B 0x0C 0x0D

Low Mem          High Mem
```

Example – demo session

`stack-smash0.c`

Control-flow Attacks

- The memory of the process contains *control data*
- In our example, this is the return address stored in the stack
- Control data dictate the flow of the program
- Overwriting control data hijacks the control flow
- Overwriting is possible, since control data are co-located with other buffers that can be overwritten due to program's vulnerabilities

Was the attack perfect?

- Not the best we could do
  - Stack was not handled correctly
  - The program crashes at the end
- Easy to carry out
  - Just change the value of the return address
  - Goal achieved (although, dirty)

Further reading

- http://eli.thegreenplace.net/2011/02/04/where-the-top-of-the-stack-is-on-x86/
- http://10kstudents.eu